STAT 432 Homework 07

Neel

All work is in Python (numpy, pandas, scikit-learn 1.9, matplotlib). Each question's code is shown with its printed output. Data files are read from `data/`.

Question 1: LDA

```
import numpy as np
import pandas as pd
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix

np.set_printoptions(precision=4, suppress=True, linewidth=140)

heart = pd.read_csv("data/saheart.csv")
heart["famhist"] = heart["famhist"].map({"Absent": 1, "Present": 2})
X = heart.drop(columns="chd").to_numpy()
y = heart["chd"].to_numpy()
names = heart.drop(columns="chd").columns
n, p = X.shape
classes = np.unique(y)
K = len(classes)

# (a) priors, class means, pooled covariance (denominator n - K)
pi = np.array([np.mean(y == k) for k in classes])                # K
mu = np.vstack([X[y == k].mean(axis=0) for k in classes])         # K x p
S = sum((X[y == k] - mu[i]).T @ (X[y == k] - mu[i]) for i, k in enumerate(classes))
Sigma = S / (n - K)                                              # p x p

print("n, p, K:", n, p, K)
print("priors:", pi)
print(pd.DataFrame(mu, index=[f"chd={k}" for k in classes], columns=names).round(4).to_string())
print(pd.DataFrame(Sigma, index=names, columns=names).round(4).to_string())

# (b) scores computed by hand
Sinv_mu = np.linalg.solve(Sigma, mu.T)                           # p x K
const = -0.5 * np.sum(mu.T * Sinv_mu, axis=0) + np.log(pi)       # K
scores = X @ Sinv_mu + const                                     # n x K
print("dims: X", X.shape, "mu", mu.shape, "Sigma", Sigma.shape, "Sinv_mu", Sinv_mu.shape, "scores", scores.shape)
yhat = classes[np.argmax(scores, axis=1)]
cm_hand = confusion_matrix(y, yhat, labels=classes)
print("Manual confusion matrix (rows observed, cols predicted):\n", cm_hand)
print("In-sample error:", np.mean(yhat != y))

# (c) sklearn
lda = LinearDiscriminantAnalysis(solver="svd", priors=pi).fit(X, y)
yhat_sk = lda.predict(X)
cm_sk = confusion_matrix(y, yhat_sk, labels=classes)
print("sklearn confusion matrix:\n", cm_sk)
print("Labels identical:", np.array_equal(yhat, yhat_sk), "| disagreements:", np.sum(yhat != yhat_sk))
print("max |sklearn means - manual|:", np.abs(lda.means_ - mu).max())
```

Output:

```
n, p, K: 462 9 2
priors: [0.6537 0.3463]
            sbp  tobacco     ldl  adiposity  famhist    typea  obesity  alcohol      age
chd=0  135.4603   2.6347  4.3442    23.9691   1.3179  52.3675  25.7375  15.9314  38.8543
chd=1  143.7375   5.5249  5.4879    28.1202   1.6000  54.4938  26.6229  19.1452  50.2938
                sbp  tobacco     ldl  adiposity  famhist    typea  obesity   alcohol       age
sbp        405.4348  14.5852  4.5812    49.1644   0.3370 -15.5877  18.9388   64.4004   95.1339
tobacco     14.5852  19.2426  0.7632     7.5381   0.0158  -2.0573   1.8335   20.5170   22.7654
ldl          4.5812   0.7632  4.0006     6.0327   0.0919   0.3446   2.6601   -2.5329    6.4789
adiposity   49.1644   7.5381  6.0327    56.7529   0.4328  -5.3096  22.7079   16.1190   60.5087
famhist      0.3370   0.0158  0.0919     0.4328   0.2258   0.0811   0.1840    0.7685    0.9974
typea      -15.5877  -2.0573  0.3446    -5.3096   0.0811  95.5656   2.6401    7.9600  -20.2783
obesity     18.9388   1.8335  2.6601    22.7079   0.1840   2.6401  17.6154    4.6894   15.6970
alcohol     64.4004  20.5170 -2.5329    16.1190   0.7685   7.9600   4.6894  598.2766   27.8859
age         95.1339  22.7654  6.4789    60.5087   0.9974 -20.2783  15.6970   27.8859  184.1321
dims: X (462, 9) mu (2, 9) Sigma (9, 9) Sinv_mu (9, 2) scores (462, 2)
Manual confusion matrix (rows observed, cols predicted):
 [[258  44]
 [ 73  87]]
In-sample error: 0.2532467532467532
sklearn confusion matrix:
 [[258  44]
 [ 73  87]]
Labels identical: True | disagreements: 0
max |sklearn means - manual|: 0.0
```

(a) With $n = 462$, $p = 9$, $K = 2$: the estimated priors are $\widehat{\pi}_0 = 302/462 = 0.654$ (no CHD) and $\widehat{\pi}_1 = 160/462 = 0.346$ (CHD). The two class centroids (a 2×9 matrix) and the 9×9 pooled covariance $\widehat{\mathbf{\Sigma}} = \frac{1}{n-K} \sum_k \sum_{i: y_i = k} \left(\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_k \right) \left(\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_k \right)^{\mathsf{T}}$ are printed above. CHD cases are on average older (50.3 vs 38.9 years), use more tobacco, and have higher LDL, adiposity and a higher rate of family history.

(b) The scores were computed as $\mathbf{X}\widehat{\mathbf{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_k - \frac{1}{2}\widehat{\boldsymbol{\mu}}_k^{\mathsf{T}}\widehat{\mathbf{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_k + \log \widehat{\pi}_k$. Dimensions check out: $\mathbf{X}$ is 462×9, $\widehat{\mathbf{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}$ is 9×2, so the score matrix is 462×2. In-sample confusion matrix (rows observed, columns predicted):

	pred 0	pred 1
obs 0	258	44
obs 1	73	87

In-sample misclassification rate $= 117/462 = 0.253$.

(c) `LinearDiscriminantAnalysis(solver="svd")` with the empirical priors gives the identical confusion matrix (258, 44 / 73, 87), and all 462 fitted labels agree with the manual ones. The results match: the SVD solver uses the same priors, centroids and pooled within-class covariance, so the discriminant scores differ only by numerical round-off.

Question 2: QDA and marginal screening

```
import numpy as np
import pandas as pd
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.metrics import confusion_matrix
import warnings

train = pd.read_csv("data/zip-train.csv.gz", header=None).to_numpy()
test = pd.read_csv("data/zip-test.csv.gz", header=None).to_numpy()
ytr, Xtr = train[:, 0].astype(int), train[:, 1:]
yte, Xte = test[:, 0].astype(int), test[:, 1:]
```

```
print("train", Xtr.shape, "test", Xte.shape)

var = Xtr.var(axis=0, ddof=1)
order = np.argsort(var)[::-1]
labels = np.arange(10)

for m in (40, 10):
    idx = order[:m]
    with warnings.catch_warnings(record=True) as w:
        warnings.simplefilter("always")
        qda = QuadraticDiscriminantAnalysis(tol=1e-8).fit(Xtr[:, idx], ytr)
        pred = qda.predict(Xte[:, idx])
        msgs = sorted({str(x.message)[:90] for x in w})
    cm = confusion_matrix(yte, pred, labels=labels)
    err = np.mean(pred != yte)
    train_err = np.mean(qda.predict(Xtr[:, idx]) != ytr)
    print(f"\n=== {m} predictors (pixel columns, 1-based: {sorted((idx + 1).tolist())}) ===")
    print("warnings:", msgs)
    print(pd.DataFrame(cm, index=[f"obs {k}" for k in labels], columns=[f"pred {k}" for k in labels]).to_string())
    print(f"test error = {err:.4f}   train error = {train_err:.4f}")
    print("per-class test error:", np.round(1 - np.diag(cm) / cm.sum(1), 3))
    # covariance diagnostics: smallest eigenvalue / condition number per class
    conds = []
    for k in labels:
        ev = np.linalg.eigvalsh(np.cov(Xtr[ytr == k][:, idx], rowvar=False))
        conds.append(ev.max() / ev.min())
    print("class sizes:", np.bincount(ytr))
    print("per-class cov condition numbers:", np.round(conds, 1))

print("\nparams per class: 40 ->", 40 + 40 * 41 // 2, " 10 ->", 10 + 10 * 11 // 2)
```

Output:

```
train (7291, 256) test (2007, 256)

=== 40 predictors (pixel columns, 1-based: [22, 27, 38, 43, 53, 54, 56, 57, 60, 69, 75, 76, 91, 92, 104, 105, 107, 120, 121, 123,
136, 137, 139, 152, 153, 154, 168, 169, 184, 185, 200, 201, 204, 213, 214, 219, 220, 230, 234, 235]) ===
warnings: []
       pred 0  pred 1  pred 2  pred 3  pred 4  pred 5  pred 6  pred 7  pred 8  pred 9
obs 0     319       0      24       6       2       2       1       0       5       0
obs 1       0     238       5       0      11       0       7       0       3       0
obs 2       8       0     168       8       8       3       0       0       3       0
obs 3       5       0       4     128       0      14       0       0      15       0
obs 4       0       2      13       1     150       1       0       1       7      25
obs 5      12       0       3       5       1     131       0       0       6       2
obs 6       0       0       8       0       1       8     148       0       5       0
obs 7       1       0       6       1       6       0       0     124       2       7
obs 8       1       0      10      12       3       2       0       0     137       1
obs 9       0       0       1       2       7       1       1       4       9     152
test error = 0.1555   train error = 0.0833
per-class test error: [0.111 0.098 0.152 0.229 0.25  0.181 0.129 0.156 0.175 0.141]
class sizes: [1194 1005  731  658  652  556  664  645  542  644]
per-class cov condition numbers: [1.8458000e+03 4.4652444e+06 3.3670000e+02 2.9540000e+02 4.1200000e+02
 1.9470000e+02 9.5710000e+02 1.8070000e+03 1.1870000e+02 6.3650000e+02]

=== 10 predictors (pixel columns, 1-based: [56, 105, 120, 121, 136, 185, 204, 219, 230, 235]) ===
warnings: []
       pred 0  pred 1  pred 2  pred 3  pred 4  pred 5  pred 6  pred 7  pred 8  pred 9
obs 0     298       0      11       1       1      16      28       4       0       0
obs 1       0     241      10       0       3       1       1       0       2       6
obs 2      19       2      95      14      14       3      22      21       5       3
obs 3       5       0       6     103       3      42       0       0       7       0
obs 4       1       5      14       0      95       2       4      15      17      47
obs 5      15       0       6      25       4      96       4       2       8       0
obs 6      41       0      15       4       0      18      78       7       5       2
```

```
obs 7       4       1       6       0       6       0       1     122       0       7
obs 8       1       2      11      36      12      11       4       1      74      14
obs 9       0       2       9       2      20       0       0      16      10     118
test error = 0.3423   train error = 0.2886
per-class test error: [0.17  0.087 0.52  0.38  0.525 0.4   0.541 0.17  0.554 0.333]
class sizes: [1194 1005  731  658  652  556  664  645  542  644]
per-class cov condition numbers: [ 63.7 165.1  28.1  15.   46.1  17.   41.   44.9  11.1  42.1]

params per class: 40 -> 860  10 -> 65
```

(a) With the 40 highest-variance pixels, QDA fits without error and the test confusion matrix is printed above (rows observed digit, columns predicted digit). **Test misclassification rate = 0.156** (training error 0.083).

(b) Yes, the method works. Ordinary QDA failed on all 256 pixels because many pixels (especially near the border) are nearly constant within a class, so each class's 256×256 covariance matrix is singular. The 40 highest-variance pixels sit in the centre of the image where digits actually differ, so they vary within every class. Each class has 542–1,194 training images, far more than the $40 + 40 \cdot 41/2 = 860$ parameters per class, so every class covariance is invertible. Digit 1 is the weak spot: a "1" is a thin stroke, many of the selected pixels are almost always blank for it, and its covariance has a condition number of about 4×10^6. It is still invertible and its test error is low (9.8%). The result is also clearly better than LDA on the same 40 pixels (test error 0.234), so letting each digit have its own covariance (its own stroke-shape variability) helps. The gap between training (0.083) and test (0.156) error shows the variance cost of the 8,600 covariance parameters.

(c) With only the 10 highest-variance pixels, the **test error rises to 0.342**, more than double the 40-pixel error. The covariance estimates are now very stable (65 parameters per class, condition numbers below 200), but the model is heavily biased: ten pixels cannot carry enough shape information to separate ten digits. Training error is also high (0.289), which marks underfitting rather than overfitting. The worst classes (2, 4, 6, 8, each above 50% error) are confused with digits that share strokes in those few pixels (6→0, 8→3, 4→9). Marginal-variance screening is also unsupervised: a pixel can vary a lot without separating classes. Going from 40 to 10 pixels trades a small drop in variance for a large rise in bias, so the error gets worse.

Question 3: Compare LDA and QDA

Random seed: **432** (`numpy.random.default_rng(432)`, one generator for the whole study).

```python
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis

SEED = 432
rng = np.random.default_rng(SEED)
rhos = [0, 0.2, 0.4, 0.6, 0.8, 0.9]
p, n_tr, n_te, R = 10, 100, 1000, 50


def gen(n, rho):
    """n observations per class; returns X (2n x 10), y in {1,2}."""
    S1 = np.array([[1, rho], [rho, 1]])
    S2 = np.array([[1, -rho], [-rho, 1]])
    X1 = np.hstack([rng.multivariate_normal([-1, 0], S1, n), rng.standard_normal((n, p - 2))])
    X2 = np.hstack([rng.multivariate_normal([1, 0], S2, n), rng.standard_normal((n, p - 2))])
    return np.vstack([X1, X2]), np.repeat([1, 2], n)


def one_run(rho):
    Xtr, ytr = gen(n_tr, rho)
    Xte, yte = gen(n_te, rho)
    lda = LinearDiscriminantAnalysis(priors=[0.5, 0.5]).fit(Xtr, ytr)
```

```
        qda = QuadraticDiscriminantAnalysis(priors=[0.5, 0.5]).fit(Xtr, ytr)
        return np.mean(lda.predict(Xte) != yte), np.mean(qda.predict(Xte) != yte)


    # (a) one dataset per rho
    single = pd.DataFrame([(r, *one_run(r)) for r in rhos], columns=["rho", "LDA", "QDA"])
    print("(a) single dataset per rho, seed", SEED)
    print(single.round(4).to_string(index=False))

    # (b) 50 repetitions per rho
    res = {r: np.array([one_run(r) for _ in range(R)]) for r in rhos}
    summ = pd.DataFrame({
        "rho": rhos,
        "LDA_mean": [res[r][:, 0].mean() for r in rhos],
        "QDA_mean": [res[r][:, 1].mean() for r in rhos],
        "LDA_se": [res[r][:, 0].std(ddof=1) / np.sqrt(R) for r in rhos],
        "QDA_se": [res[r][:, 1].std(ddof=1) / np.sqrt(R) for r in rhos],
        "QDA_wins": [np.mean(res[r][:, 1] < res[r][:, 0]) for r in rhos],
    })
    print("\n(b) 50 reps")
    print(summ.round(4).to_string(index=False))
    summ.to_csv("q3_summary.csv", index=False)

    # Bayes errors (reference): LDA-optimal rule uses x1 only (pooled cov is diagonal);
    from scipy.stats import norm
    print("\nBest linear (x1<0) error:", round(norm.cdf(-1), 4))

    fig, ax = plt.subplots(figsize=(6.5, 4.2))
    for col, lab, c, m in [("LDA", "LDA", "#2a6fdb", "o"), ("QDA", "QDA", "#d1495b", "s")]:
        ax.errorbar(summ.rho, summ[f"{col}_mean"], yerr=1.96 * summ[f"{col}_se"], label=lab,
                    color=c, marker=m, capsize=3, lw=1.8)
    ax.set_xlabel(r"True correlation $\rho$")
    ax.set_ylabel("Average test misclassification rate")
    ax.set_title(f"LDA vs QDA across data-generating distributions\n(50 reps per $\\rho$, 100 train / 1000 test per class,
seed {SEED})", fontsize=10)
    ax.set_xticks(rhos)
    ax.grid(alpha=0.3)
    ax.legend(frameon=False)
    fig.tight_layout()
    fig.savefig("q3_error_vs_rho.png", dpi=200)
```

Output:

```
(a) single dataset per rho, seed 432
 rho    LDA    QDA
 0.0 0.1710 0.2010
 0.2 0.1715 0.1815
 0.4 0.1575 0.1715
 0.6 0.1535 0.1610
 0.8 0.1815 0.1495
 0.9 0.1815 0.1030

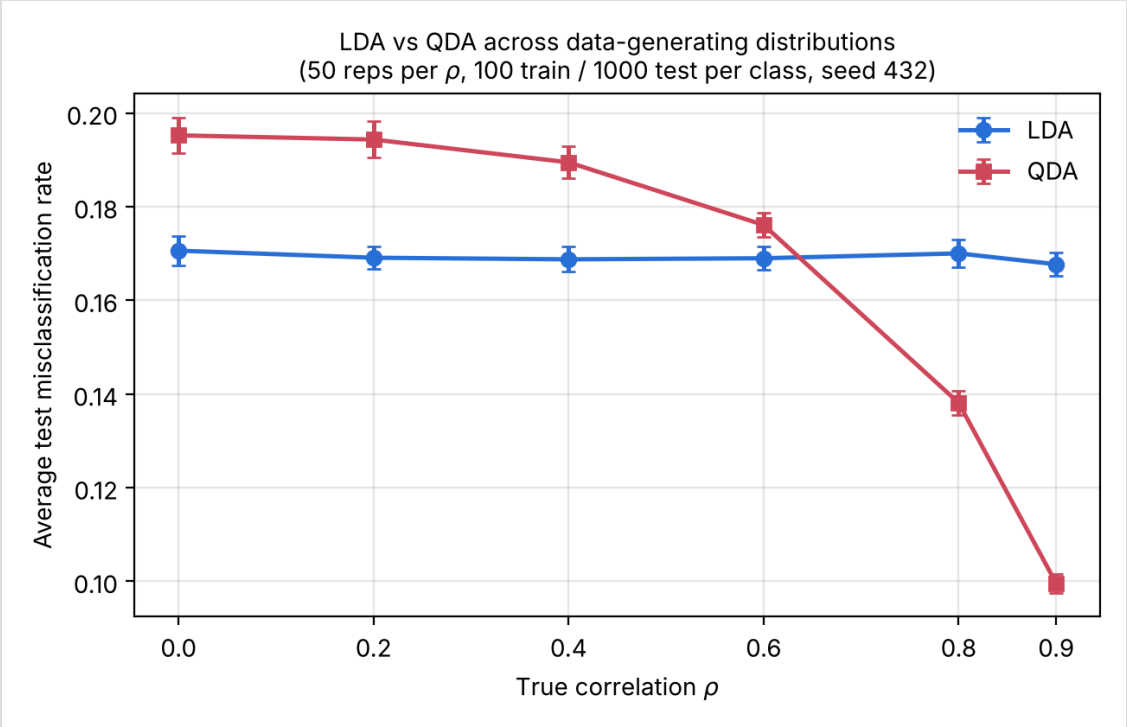
(b) 50 reps
 rho  LDA_mean  QDA_mean  LDA_se  QDA_se  QDA_wins
 0.0    0.1706    0.1952  0.0016  0.0020      0.00
 0.2    0.1691    0.1943  0.0012  0.0020      0.00
 0.4    0.1688    0.1895  0.0014  0.0018      0.00
 0.6    0.1690    0.1761  0.0013  0.0013      0.24
 0.8    0.1700    0.1381  0.0015  0.0014      1.00
 0.9    0.1677    0.0996  0.0013  0.0010      1.00

Best linear (x1<0) error: 0.1587
```

(a) One dataset per ρ (same training and test data for both methods within each setting):

ρ	0	0.2	0.4	0.6	0.8	0.9
LDA test error	0.171	0.172	0.158	0.154	0.182	0.182
QDA test error	0.201	0.182	0.172	0.161	0.150	0.103

(b) Average test error over 50 repetitions per ρ (error bars are ± 1.96 SE):



ρ	0	0.2	0.4	0.6	0.8	0.9
LDA	0.171	0.169	0.169	0.169	0.170	0.168
QDA	0.195	0.194	0.190	0.176	0.138	0.100
share of reps QDA < LDA	0.00	0.00	0.00	0.24	1.00	1.00

(c) For small ρ (0 to 0.4), LDA is better by about 2–2.5 percentage points and wins in every repetition. Near $\rho = 0.6$ the two methods are close. For large ρ, QDA wins clearly: 0.138 vs 0.170 at $\rho = 0.8$ and 0.100 vs 0.168 at $\rho = 0.9$.

Why. The two classes have covariances with opposite-sign correlations, so the pooled covariance is $\text{diag}\,(1, \ldots, 1)$ for every ρ. LDA therefore always estimates essentially the same linear rule, roughly "classify by the sign of x_1", whose best possible error is $\Phi(-1) = 0.159$ whatever ρ is. That is why the LDA curve is flat at about 0.169. Once $\rho \neq 0$, LDA's equal-covariance assumption is wrong, so it has *bias* that grows with ρ. It ignores the information carried by the sign of the x_1–x_2 correlation. QDA's model is correct for every ρ, so it has no such bias, but it estimates two full 10×10 covariances ($2 \times 55 = 110$ covariance parameters vs 55 for LDA) from only 100 observations per class. That extra *variance* costs about 2.5 points when ρ is small, because there is little curvature to gain. As ρ grows, the Bayes boundary becomes strongly quadratic (the classes form an "X" in the (x_1, x_2) plane), and QDA's bias reduction outweighs its variance. Hence the crossover near $\rho \approx 0.6$.

Noise covariates. The eight noise variables carry no class information, but both methods still estimate means, variances and covariances for them. Going from 2 to 10 covariates adds $8 \times 2 = 16$ mean parameters and $55 - 3 = 52$ covariance entries for LDA, and $2 \times 52 = 104$ covariance entries for QDA. Every estimate is noisy, so the fitted boundaries pick up spurious tilts (LDA) and spurious curvature (QDA) in the noise directions, which adds variance to the test error. This penalty is about twice as large for QDA. As a quick check, I reran the same design with only (X_1, X_2) (50 reps, seed 1). LDA's error dropped to about 0.158 (close

to $\Phi(-1)$, versus 0.169 with noise). At $\rho = 0$, QDA was only 0.001 worse than LDA (versus 0.025 with noise), and QDA already won at $\rho = 0.6$ (0.150 vs 0.158). So the noise covariates are what make QDA's variance cost large and push the crossover toward higher ρ.

Question 4: LDA/QDA analysis skill

Skill definition, file size, and explicit invocation

The skill is a single file, `lda-qda-report/SKILL.md`, **3,779 bytes** (under the 5,000-byte limit). Its complete contents are appended at the end. Explicit-only invocation is set in the frontmatter with `disable-model-invocation: true`, which tells Claude Code never to load the skill automatically. It runs only when the user types `/lda-qda-report`. The skill contains no dataset names, variable names or results: the dataset, response and predictors are supplied in the prompt.

Prompt used to apply the skill:

> /lda-qda-report Analyze `data/saheart.csv`. Response: `chd` (0/1). Predictors: all nine other columns as numeric, with `famhist` coded 1 = Absent, 2 = Present. Seek the best predictive performance among the LDA/QDA methods the skill allows, including ordinary LDA and QDA as comparisons. Use Python and random seed 432.

Review. I checked the agent's analysis using only training and CV results: the test split happens before any fitting; one set of 10 stratified folds is reused for every candidate; standardization and PCA sit inside the per-fold pipeline; and the selection rule (one SE toward fewer parameters) was fixed in the skill before any results were seen. No revisions to the skill were needed. The analysis code (`q4_saheart_analysis.py`, `q4_make_report.py`) is included with the submission.

One-page analysis report

The one-page analysis report is on the next page. The complete `SKILL.md` follows it.

LDA/QDA classification of coronary heart disease (SAheart)

Response *chd* (0/1); p = 9 numeric predictors (famhist coded 1/2); n = 462; random seed 432.

Setup

A stratified 25% test set (116 obs.) was reserved before any modelling; the remaining 346 observations (226 no CHD / 120 CHD) were split once into 10 stratified folds used for every candidate. Candidates: ordinary LDA and QDA on all predictors; PCA (standardize + first d components, d = 1–9, fitted inside each fold) followed by LDA or QDA; and a blend $w \cdot P_{LDA} + (1-w) \cdot P_{QDA}$, w = 0, 0.1, …, 1. Empirical priors; selection by misclassification rate with a one-SE rule toward fewer parameters.

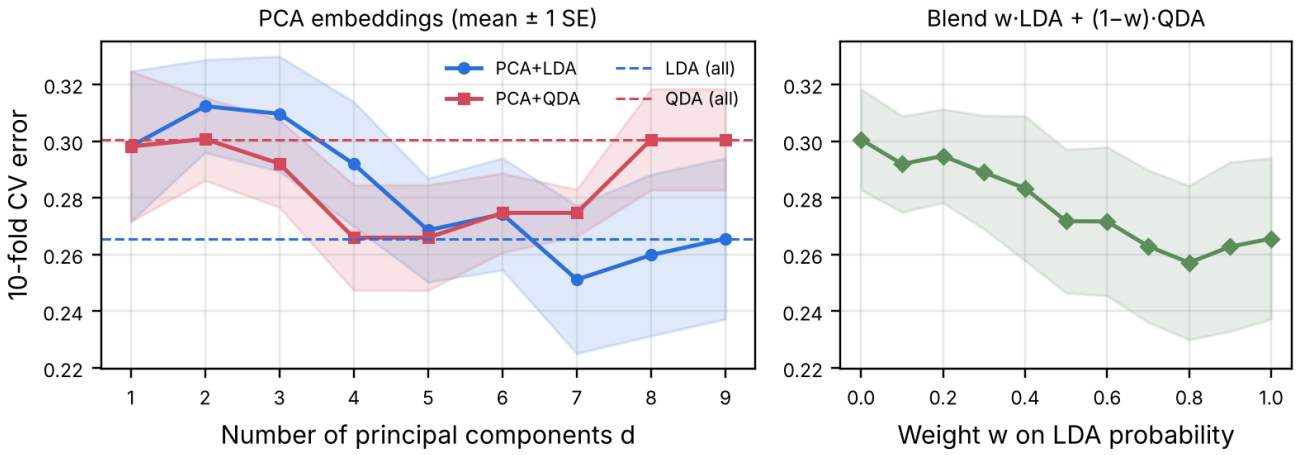


Figure: 10-fold CV misclassification rate (bands = ±1 SE). Dashed lines: ordinary LDA and QDA on all 9 predictors.

Cross-validation summary (best setting per family)

Model family	Best setting	Params*	CV error	SE	
Ordinary LDA	all 9	63	0.266	0.028	
Ordinary QDA	all 9	108	0.301	0.018	
PCA + LDA	d=7	42	0.251	0.026	min CV
PCA + QDA	d=4	28	0.266	0.019	selected
Blend (LDA, QDA)	w=0.8	171	0.257	0.027	

*Class means + covariance entries in the space the classifier uses (PCA loadings not counted).

Final model

Lowest CV error: PCA + LDA (d = 7), 0.251 (SE 0.026). One-SE threshold 0.277; the model with fewest parameters under it is **PCA + QDA with d = 4 components** (CV 0.266). Refit on all 346 training observations: training error 0.240; **test error 0.276** (32/116), exact 95% CI [0.197, 0.367]. Test confusion matrix (rows observed, columns predicted, 0/1): no CHD [64, 12]; CHD [20, 20] — CHD sensitivity 0.50, specificity 0.84.

Findings

- Ordinary QDA (CV 0.301) is worse than ordinary LDA (0.266): with about 120 CHD cases, two full 9×9 covariance matrices add variance that outweighs any gain from unequal class covariances.
- Reducing dimension helps QDA most: with 4 components its error matches ordinary LDA while estimating far fewer parameters, a direct bias–variance gain. Very small d (1–3) discards class information and raises error for both methods.
- Blending moves smoothly from QDA (w = 0) to LDA (w = 1) with a shallow minimum near w = 0.8, i.e. the data favour mostly-linear boundaries.
- All best settings lie within one SE of each other (≈0.25–0.27): no extension beats plain LDA by more than one SE.

Limitations

- One train/test split and one set of folds; CV SEs (≈0.02–0.03) are large relative to differences between models.
- Small test set (116): the CI for test error spans about 17 points. Choosing among ~30 settings makes the minimum CV error optimistic.
- LDA/QDA assume Gaussian classes; famhist is binary and alcohol/tobacco are skewed. Sensitivity to CHD is low at the 0.5 threshold.

Appendix: complete SKILL.md

File: `lda-qda-report/SKILL.md` (3,779 bytes).

```
---
name: lda-qda-report
description: Run a leakage-free LDA/QDA classification analysis and write a one-page statistical report. Use ONLY when the user explicitly invokes lda-qda-report.
disable-model-invocation: true
---

# LDA/QDA Analysis Report

The user supplies a dataset, a categorical response, and numeric predictors (with any coding). Never assume these; ask if missing. Work in R or Python as requested (default Python).

## Rules
- Use only classifiers built from LDA, QDA, or a blend of their predicted class probabilities. No other model families.
- Set and report one random seed. Use misclassification rate as the only selection criterion.
- Reserve a stratified test set (default 25%) BEFORE any exploration or tuning. Touch it exactly once, at the end.
- Use one fixed set of stratified K-fold CV splits of the training set (default K = 10) for every candidate and every setting.
- Fit all preprocessing (centering, scaling, PCA) inside each fold on that fold's training part only (e.g. a pipeline).
- Use empirical class priors unless the user specifies priors.
- Never change the plan after seeing test results.

## Candidate models
1. Ordinary LDA on all predictors (always included).
2. Ordinary QDA on all predictors (always included).
3. PCA + LDA and PCA + QDA: standardize, then keep the first d components, d in 1..p (or a grid of at most 10 values if p is large).
4. Blend: P = w * P_LDA + (1 - w) * P_QDA, w in {0, 0.1, ..., 1}; predict the class with the largest P.
Optionally blend after PCA if steps 3 and 4 both look useful. Keep the candidate list short.

## Procedure
1. Describe the data briefly: n, p, class counts; flag constant or near-collinear predictors and classes too small for QDA (n_k <= p).
2. For each candidate and setting, compute the CV error mean and its standard error (SD across folds / sqrt(K)).
3. Select the setting with the lowest mean CV error within each family. Pick the overall model with the lowest mean CV error; if several are within one SE of the best, prefer the simplest (fewest parameters) and say so.
4. Refit the selected model and its preprocessing on the full training set. Evaluate once on the test set: error, confusion matrix (observed rows, predicted columns), and a 95% binomial CI for the error.
5. Sanity checks: matrix dimensions, no warnings about singular covariance left unexplained, in-sample error not far below CV error without comment.

## Report (one printed page, letter, max)
Produce a single-page PDF (and the source). Readable fonts (>= 9 pt). Sections in this order:
- **Title line**: dataset, response, n, p, seed.
- **Setup** (2-3 sentences): split, CV folds, preprocessing, candidates.
- **Figure**: CV error (mean +/- 1 SE) versus tuning setting, one line per family (PCA+LDA, PCA+QDA vs d; blend vs w on a secondary panel or axis), with horizontal reference lines for ordinary LDA and QDA. Label axes; one panel, or two side by side.
- **Table**: one row per family at its best setting: model, setting, approximate parameter count, CV error, SE. Mark the selected row.
- **Final model results**: selected model, training (resubstitution) error, test error with 95% CI, test confusion matrix.
- **Findings** (3-5 bullets): what the CV comparison shows about the covariance assumption and bias-variance trade-off; whether the extension beat plain LDA/QDA by more than one SE.
- **Limitations** (2-3 bullets): e.g. single split, small test set, normality, CV noise, selection optimism.
Use plain wording, three significant digits for errors, and no raw code in the report. Check the page count before finishing; shorten text, not the figure, if it overflows.
```

Deliver

Return the PDF, the script that reproduces it, and a short chat summary: selected model, CV error, test error.